ACOUSTIC HUB
Hệ thống Giám sát & Điều khiển Âm thanh tại biên

1. Tổng quan:

1.1: Vấn đề hiện nay:

- Các hệ thống giám sát và Smart home hiện hành phụ thuộc vào điện toán đám mây (Cloud Computing) để phân tích dữ liệu âm thanh.
- Kiến trúc này phát sinh ba rủi ro cốt lõi:

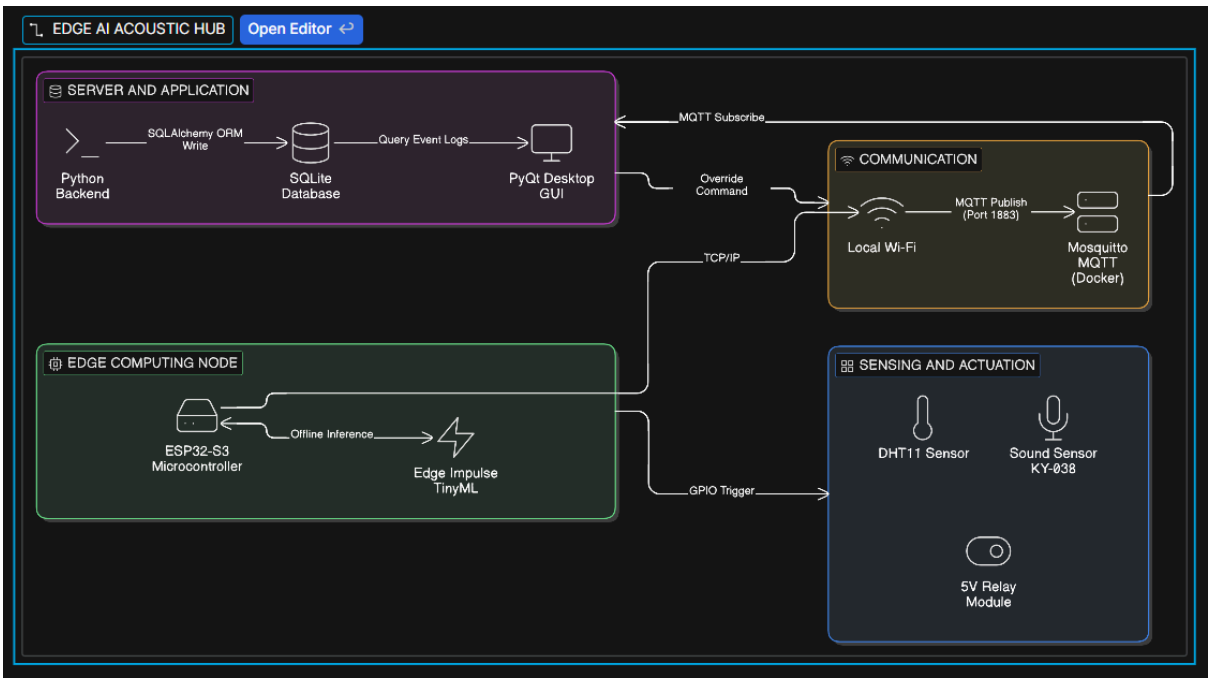
+ Độ trễ truyền tải mạng.
+ Rủi ro rò rỉ dữ liệu riêng tư.
+ Mất khả năng hoạt động khi gián đoạn kết nối Internet.

1.2: Giải pháp:

Phát triển 1 giải pháp điện toán tại biên có khả năng:

- Thu thập tín hiệu analog liên tục từ môi trường (âm thanh, nhiệt độ, độ ẩm).
- Chạy mô hình TinyML offline 100% để phân loại âm thanh mục tiêu (tiếng vỗ tay, còi báo động, …).
- Điều khiển thiết bị ngoại vi với độ trễ nhỏ.
- Đồng bộ trạng thái về máy chủ cục bộ qua giao thức bản tin nhẹ.

2. Kiến trúc hệ thống:



- ## Sensing and Actuation:

Lớp giao tiếp trực tiếp với môi trường vật lý, chịu trách nhiệm thu thập dữ liệu thô và thực thi các lệnh vật lý.

- **Input:**
+ **Sound Sensor KY-038**: Liên tục thu thập sóng âm thanh từ môi trường và xuất ra tín hiệu Analog.
+ **DHT11 Sensor**: Thu thập dữ liệu nhiệt độ và độ ẩm phòng, giao tiếp qua chuẩn Digital 1 dây.

- **Output:**
+ **5V Relay Module:** Công tắc điện tử, nhận tín hiệu kích hoạt từ vi điều khiển để đóng/Ngắt các thiết bị điện công suất lớn.

- ## Edge Computing Node:

Xử lý khối lượng tính toán lớn mà không cần đẩy dữ liệu lên Cloud

- **ESP32-S3 Controller:**
+ Trung tâm điều phối.

+ Để xử lý tín hiệu Analog liên tục từ KY-038 mà không làm nghẽn vi xử lý, phần mềm (Firmware) áp dụng các kỹ thuật lập trình C/C++ chuyên sâu.
+ Sử dụng con trỏ (pointer) để cấp phát và quản lý linh hoạt các mảng bộ nhớ đệm (buffer) giúp vi điều khiển vừa đọc dữ liệu, vừa tính toán mượt mà.

- **Edge Impulse TinyML:**
+ ESP32-S3 lấy các mảng dữ liệu âm thanh đã được chuẩn hóa và đẩy vào mạng Neural Network (được nhúng sẵn trong code C++).
+ Quá trình suy luận ngoại tuyến (Offline Inference) này diễn ra hoàn toàn trên chip để nhận diện các mẫu âm thanh (như tiếng vỗ tay), từ đó ra quyết định bật/tắt Relay ngay lập tức với độ trễ gần như bằng không.

● Communication:

Định tuyến các luồng sự kiện giữa phần cứng và phần mềm

- **Local Wifi (TCP/IP):**

Kết nối không dây nội bộ, đảm bảo tính bảo mật và không phụ thuộc vào đường truyền Internet bên ngoài.

- **Mosquitto MQTT (Docker):**
+ Trạm trung chuyển bản tin (Broker) chuẩn công nghiệp.
+ Việc đóng gói Mosquitto vào Docker container giúp môi trường mạng luôn được cô lập, ổn định và dễ dàng khởi tạo (deploy) trên bất kỳ máy chủ nào.
+ Broker này mở cổng 1883 để nhận các bản tin Publish từ ESP32-S3 và phân phối chúng đến các thiết bị đang Subscribe.

● Server and Application:

Lưu trữ dữ liệu chuỗi thời gian, xử lý logic nghiệp vụ và tương tác với người dùng.

- **Backend:**
+ Vai trò là MQTT Client chạy ngầm, lắng nghe (Subscribe) các topic từ Broker.
+ Khi có sự kiện âm thanh mới gửi tới, thay vì dùng truy vấn SQL thuần dễ sinh lỗi, hệ thống dùng **SQLAlchemy (ORM)** để tự động ánh xạ (map) cấu trúc dữ liệu JSON thành các đối tượng và ghi (Write) một cách an toàn vào cơ sở dữ liệu.

- **SQLite Database:**

Cơ sở dữ liệu quan hệ gọn nhẹ, lưu trữ toàn bộ lịch sử trạng thái môi trường và log sự kiện thiết bị.

- PyQt Desktop GUI:

+ Giao diện điều khiển trung tâm. Ứng dụng này sẽ truy vấn (Query) dữ liệu từ SQLite để hiển thị bảng theo dõi (Event Logs) trực quan.
+ Cung cấp tính năng "Override Command" – cho phép người dùng click trên màn hình để xuất bản (Publish) một lệnh MQTT đi ngược hệ thống, qua Broker và xuống thẳng ESP32-S3 để điều khiển Relay thủ công.

3. Thông số công nghệ:

- **Phát triển Firmware:**
- Language: C/C++
- Framework: ESP-IDF

- **Model:** Nền tảng Edge Impulse

- **Server**: Linux Environment, quản lý Container qua Docker và Docker Compose

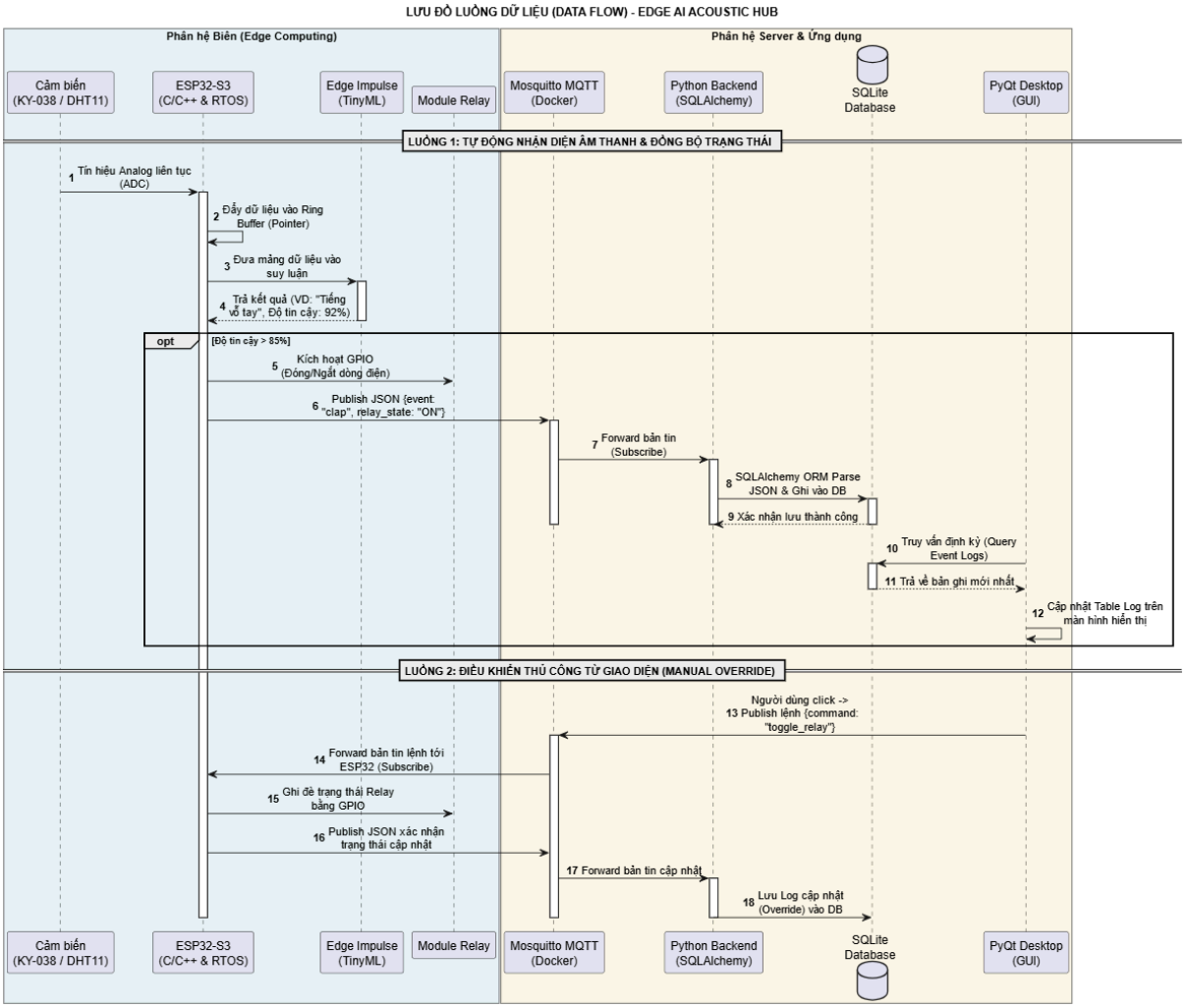
- **Backend&Database:** Python, SQLAlchemy

- **User Interface**: PyQT5

4. Quản lý tài nguyên và Hiệu năng:

- **Xử lý ADC và DMA**: Sử dụng cơ chế DMA để thu thập dữ liệu âm thanh từ KY-038.
- **Memory Management**: Áp dụng kỹ thuật cấp phát bộ nhớ động bằng con trỏ (Pointer) trong C/C++ để duy trì một Ring Buffer. Dữ liệu âm thanh mới liên tục ghi đè lên dữ liệu cũ nhất, đảm bảo mô hình AI luôn có một cửa sổ khung thời gian (Time window) nhất định để phân tích mà không gây tràn RAM (Stack/Heap overflow).

5. Lưu đồ luồng dữ liệu



Luồng 1 (Bước 1 - 12): Tự động Nhận diện Âm thanh & Đồng bộ Trạng thái

- **Giai đoạn Thu thập & Quản lý bộ nhớ (Bước 1 - 2):** * Cảm biến liên tục đẩy tín hiệu Analog về. Điểm sáng kỹ thuật ở đây nằm ở Bước 2. Thay vì lưu trữ thô bạo làm tràn RAM, vi điều khiển sử dụng các kỹ thuật C/C++ (như con trỏ - pointer) để cấp phát và quản lý một **Ring Buffer** (Bộ đệm vòng). Dữ liệu mới sẽ trượt đè lên dữ liệu cũ, tạo ra một cửa sổ thời gian cố định.

- **Giai đoạn Xử lý AI tại biên - Edge Inference (Bước 3 - 4):**

ESP32-S3 bốc mảng dữ liệu từ Ring Buffer đưa vào mô hình Edge Impulse. Quá trình tính toán ma trận này diễn ra 100% offline. AI sẽ trả về nhãn (Label) và độ tin cậy (Confidence score).

- **Giai đoạn Ra quyết định & Kích hoạt (Bước 5 - 6):**

Vùng `opt` (Option/Condition) chỉ ra rằng: Chỉ khi độ tin cậy vượt ngưỡng 85%, ESP32-S3 mới xuất mức logic ra chân GPIO để đóng/ngắt Relay ngay tức thì. Cùng lúc đó, nó đóng gói một bản tin JSON siêu nhẹ để báo cáo trạng thái lên mạng.

- **Giai đoạn Đồng bộ Server & Giao diện (Bước 7 - 12):**
+ Mosquitto Broker (chạy cách ly an toàn trong Docker) làm nhiệm vụ trung chuyển (Forward) bản tin JSON đến Python Backend.
+ Tại Bước 8, Backend sử dụng cơ chế **SQLAlchemy ORM** để "dịch" bản tin JSON thành các đối tượng dữ liệu và ghi an toàn vào SQLite Database, tránh các lỗi truy vấn SQL thủ công.
+ Cuối cùng, giao diện PyQt trên máy tính liên tục truy vấn SQLite và hiển thị log mới nhất lên màn hình giám sát.

Luồng 2 (Bước 13 - 18): Điều khiển thủ công từ giao diện (Manual Override)

- **Lệnh từ người dùng (Bước 13 - 14):**

Vận hành viên click nút trên giao diện PyQt. Lập tức một lệnh `toggle_relay` được đóng gói và Publish lên MQTT Broker, sau đó Broker đẩy thẳng xuống ESP32-S3 (đang Subscribe topic nhận lệnh).

- **Thực thi lệnh ưu tiên (Bước 15):**

ESP32-S3 nhận lệnh, tạm ngưng vòng lặp AI, và lập tức thay đổi trạng thái chân GPIO điều khiển Relay theo ý người dùng.

- **Vòng lặp phản hồi - Feedback Loop (Bước 16 - 18):**

Để đảm bảo tính toàn vẹn dữ liệu (Data Integrity), sau khi thực thi xong, ESP32-S3 bắt buộc phải gửi lại một bản tin JSON xác nhận. Python Backend bắt bản tin này và dùng SQLAlchemy ghi đè (Override) vào log dữ liệu, giúp trên giao diện hiển thị chính xác trạng thái thực tế của thiết bị vật lý.

KẾ HOẠCH TRIỂN KHAI:

PHASE 1: Xây dựng Firmware & Thu thập dữ liệu (Sensing & Data Acquisition)

Mục tiêu: Đảm bảo vi điều khiển đọc được dữ liệu cảm biến analog liên tục mà không làm nghẽn hệ thống.

Công việc cốt lõi:

- Thiết lập môi trường phát triển **ESP-IDF** (C/C++).
- Cấu hình bộ biến đổi ADC (Analog-to-Digital Converter) cho chân GPIO nối với cảm biến âm thanh KY-038.
- Thiết kế cấu trúc dữ liệu **Ring Buffer** sử dụng con trỏ (Pointer) để quản lý bộ nhớ đệm luồng âm thanh.
- Viết script đẩy dữ liệu âm thanh thô qua cổng Serial lên nền tảng thu thập.

Đầu ra kỳ vọng (Deliverable): Mã nguồn C/C++ đọc dữ liệu ADC mượt mà và file dataset âm thanh thô (đã thu thập đủ các tiếng ồn, tiếng vỗ tay).

PHASE 2: Phát triển mô hình Edge AI (TinyML Training & Inference)

Mục tiêu: Chạy thành công mạng nén thần kinh (Neural Network) 100% offline trên ESP32-S3.

Công việc cốt lõi:

- Upload dữ liệu lên **Edge Impulse**, tiến hành gán nhãn (Labeling) và làm sạch dữ liệu.
- Thiết lập Pipeline: Trích xuất đặc trưng phổ âm (MFCC) và cấu hình các lớp (Layers) của mạng Neural Network.
- Huấn luyện (Train) mô hình, đánh giá độ chính xác và tinh chỉnh (Fine-tuning) cho đến khi đạt *Accuracy > 85%*.
- Xuất mô hình thành thư viện C++, nhúng (embed) vào ESP-IDF. Viết logic kích hoạt Module Relay qua GPIO khi nhận diện thành công.

Đầu ra kỳ vọng (Deliverable): Hệ thống phần cứng hoạt động độc lập (Offline Standalone), vỗ tay đúng nhịp là Relay đóng/ngắt độ trễ < 500ms.

PHASE 3: Xây dựng Hạ tầng Mạng & IoT (Network & Broker Setup)

Mục tiêu: Số hóa và truyền tải trạng thái vật lý lên môi trường mạng nội bộ.

Công việc cốt lõi:

- Viết file `docker-compose.yml` để ảo hóa và khởi chạy **Eclipse Mosquitto MQTT Broker** trên môi trường Linux.
- Tích hợp thư viện MQTT Client vào firmware ESP-IDF.
- Viết hàm kết nối Wi-Fi, tự động Reconnect khi rớt mạng.
- Đóng gói trạng thái hệ thống thành bản tin JSON và Publish lên Broker.

Đầu ra kỳ vọng (Deliverable): Broker nhận được bản tin JSON `{"event": "clap", "relay_state": "ON"}` mỗi khi AI trên ESP32-S3 kích hoạt Relay.

PHASE 4: Phát triển Backend & Giao diện (Software & GUI Dashboard)

Mục tiêu: Xây dựng phần mềm trung tâm để lưu trữ log sự kiện và cho phép con người giám sát, điều khiển.

Công việc cốt lõi:

- Khởi tạo project **Python**. Dùng **SQLAlchemy (ORM)** định nghĩa cấu trúc bảng (Tables) cho SQLite/PostgreSQL.
- Viết tiến trình ngầm (Daemon process) Subscribe topic MQTT, bóc tách JSON và tự động ghi (Write) vào Database.
- Thiết kế giao diện Desktop bằng **PyQt**, tạo các Table View để hiển thị Event Log theo thời gian thực (Query từ DB).
- Thêm nút "Override" trên GUI, cho phép Publish bản tin chiều ngược lại để điều khiển Relay thủ công.

Đầu ra kỳ vọng (Deliverable): Phần mềm PyQt hoàn chỉnh, có giao diện giám sát rõ ràng và khả năng điều khiển thiết bị hai chiều. File báo cáo tổng kết và Source code đẩy lên GitHub kèm README chi tiết.